

1. Real-Time Face Recognition for Campus Security

A university wants to deploy a camera-based security system at the entrance that can identify

registered students and staff in real time.

The system should:

- ☐ **Capture faces from live CCTV/video streams.**
- ☐ **Detect faces under different lighting conditions.**
- ☐ **Extract suitable facial features.**
- ☐ **Recognize registered individuals.**
- ☐ **Maintain real-time performance on a low-cost computer.**
- ☐ **Minimize false recognition.**

Design a complete OpenCV-based face recognition pipeline from image acquisition to final

identification. Explain the preprocessing, face detection, feature extraction, recognition, and

optimization techniques used. Justify how your system achieves accuracy and real-time performance.

Real-Time Face Recognition for Campus Security

1. Image Acquisition

- **Use an OpenCV camera/CCTV stream** to capture video continuously.
- **Each frame is processed to identify people entering the campus.**

2. Preprocessing

- **Resize the frame to reduce processing time.**
- **Convert the image to grayscale.**
- **Remove noise using filtering.**

- Improve poor lighting using **Histogram Equalization or CLAHE**.
- Align the detected face for better recognition.

3. Face Detection

- Use OpenCV **Haar Cascade or DNN face detector**.
- Detect multiple faces in the same frame.
- Draw a bounding box around each detected face.

4. Feature Extraction

- Crop each detected face.
- Extract important facial features using a face recognition model.
- Convert the face into a numerical **feature vector/embedding**.

5. Face Recognition

- Compare the extracted features with the registered student/staff database.
- Use **Euclidean distance or cosine similarity**.
- If the similarity is above the required threshold, identify the person.
- Otherwise, display “**Unknown Person.**”

6. Reducing False Recognition

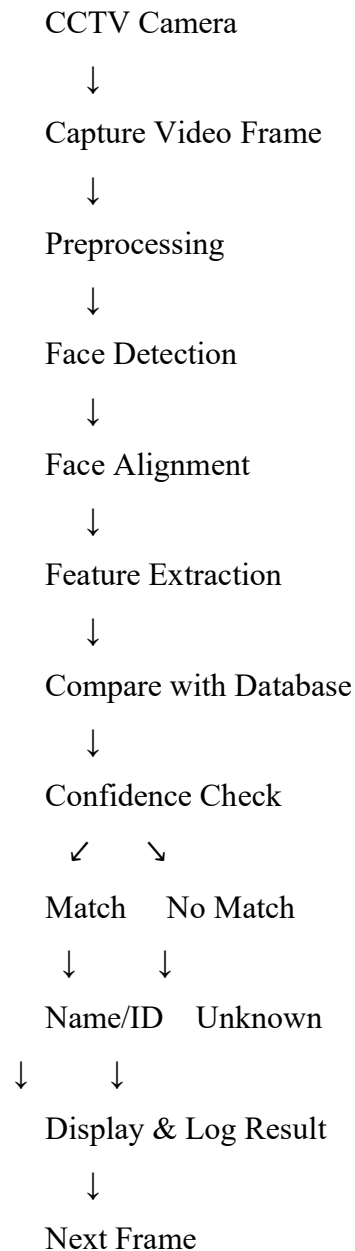
- Store multiple images of each registered person.
- Use good-quality images during registration.
- Apply face alignment and proper thresholds.
- Confirm recognition across multiple frames before accepting the identity.

7. Real-Time Optimization

- Resize video frames.
- Process only the **Region of Interest (ROI)** at the entrance.
- Skip some frames when necessary.

- Use lightweight OpenCV DNN models.
- Track already detected faces instead of detecting them repeatedly.

Flowchart



Conclusion

- The proposed OpenCV system provides **accurate and real-time face recognition** by combining preprocessing, face detection, feature extraction, and database matching. Frame resizing, ROI processing, lightweight models, and tracking help achieve **fast**

performance on a low-cost computer, while confidence thresholds and multi-frame verification reduce false recognition.

2. Intelligent Traffic Violation Detection System

A traffic department wants to automatically monitor vehicles using roadside cameras.

The system should:

- ☐ **Detect vehicles from live video.**
- ☐ **Track vehicles across consecutive frames.**
- ☐ **Identify vehicles crossing a predefined line.**
- ☐ **Count vehicles accurately.**
- ☐ **Detect abnormal or unwanted movements.**
- ☐ **Display vehicle count and alerts in real time.**

Design an OpenCV-based solution incorporating video acquisition, preprocessing, object detection, motion analysis, tracking, and visualization. Explain how you would handle shadows, camera noise, dynamic backgrounds, and multiple vehicles to minimize counting errors.

1. Video Acquisition

- Use CCTV cameras to capture live traffic video.
- OpenCV VideoCapture() reads frames continuously.

2. Preprocessing

- Resize frames to improve speed.
- Apply **Gaussian Blur** to remove camera noise.
- Use grayscale and shadow removal techniques.

3. Vehicle Detection

- Use OpenCV **DNN/Object Detection** to detect cars, bikes, buses, and trucks.
- Draw bounding boxes around vehicles.

4. Vehicle Tracking

- Assign a unique ID to each vehicle.
- Use **Kalman Filter, Centroid Tracking, or SORT** to track vehicles across frames.

5. Vehicle Counting

- Draw a **virtual line** on the road.
- When a vehicle crosses the line, increase the count.
- Count each vehicle ID only once.

6. Violation Detection

Detect:

- Wrong-direction movement
- Illegal lane changes
- Sudden stopping
- Restricted-area entry

Display an alert when a violation occurs.

7. Handling Errors

- Use shadow detection to avoid counting shadows as vehicles.
- Use noise filtering for camera noise.
- Use ROI to ignore unwanted background.
- Tracking helps handle multiple vehicles and prevents duplicate counting.

Flowchart

CCTV Video



Preprocessing



Vehicle Detection



Motion Analysis



Vehicle Tracking



Line Crossing



Count / Violation



Display Alert

Conclusion: OpenCV combines detection, tracking, motion analysis, and line-crossing techniques to achieve **accurate real-time vehicle counting and traffic violation detection**.

3. Automated Document Processing System

A company receives thousands of scanned documents that may contain blurred text, uneven

illumination, noise, and different font sizes.

The system should:

- ☐ **Automatically improve document quality.**
- ☐ **Remove noise and unwanted background.**
- ☐ **Convert the document into a suitable format for OCR.**
- ☐ **Segment text regions.**
- ☐ **Extract readable text efficiently.**
- ☐ **Process a large number of documents in batch mode.**

Design an OpenCV-based image processing pipeline for this system. Explain the role of

grayscale conversion, thresholding, filtering, morphological operations, segmentation, and

geometric correction. Justify how each stage improves OCR accuracy.

Pipeline:

Input Scanned Document → Grayscale → Noise Filtering → Illumination Correction → Thresholding → Morphological Operations → Geometric Correction → Text Segmentation → OCR → Extracted Text

Role of Each Stage

1. Grayscale Conversion

- Converts the image from RGB to a single intensity channel.
- Reduces computation and makes text/background separation easier.

2. Filtering

- Uses **Gaussian/Median filtering** to remove noise, dust, and small unwanted spots.
- Produces smoother characters and improves OCR recognition.

3. Thresholding

- Converts the grayscale image into a binary image.
- **Adaptive thresholding** handles uneven illumination and different background brightness.
- Makes text clearer from the background.

4. Morphological Operations

- **Opening** removes small noise.
- **Closing** fills small gaps in characters.
- Improves the shape and connectivity of text.

5. Geometric Correction

- Detects the document boundary and corrects **rotation, skew, and perspective**.
- Ensures text lines are horizontally aligned, improving OCR accuracy.

6. Text Segmentation

- Separates the document into **paragraphs, lines, words, or text regions**.
- Contours or connected-component analysis can be used.
- Allows OCR to process only relevant text regions.

7. OCR Extraction

- The cleaned and segmented image is given to an OCR engine such as Tesseract.
- Produces readable digital text.

8. Batch Processing

- Process documents one by one automatically using OpenCV.
- Results can be stored in TXT/CSV/database format.

Conclusion

This pipeline improves OCR accuracy by **removing noise, correcting illumination and geometry, separating text from the background, and segmenting text properly** before recognition. It is suitable for processing thousands of scanned documents efficiently.

4. Real-Time Multi-Person Tracking and Attendance System

A college wants to automatically record student attendance using classroom CCTV cameras.

The system should:

- ☐ Detect multiple faces simultaneously.
- ☐ Track each person across video frames.
- ☐ Recognize registered students.
- ☐ Avoid repeatedly marking the same student.
- ☐ Handle slight head movement and temporary face disappearance.
- ☐ Operate continuously with minimum processing delay.

Design a complete OpenCV-based system using face detection, feature extraction, recognition,

and tracking. Explain how you maintain identity across frames and prevent false or duplicate attendance records.

4. Real-Time Multi-Person Tracking and Attendance System

A college wants to automatically record student attendance using classroom CCTV cameras.

The system should:

- ☐ **Detect multiple faces simultaneously.**
- ☐ **Track each person across video frames.**
- ☐ **Recognize registered students.**
- ☐ **Avoid repeatedly marking the same student.**

☐ **Handle slight head movement and temporary face disappearance.**

☐ **Operate continuously with minimum processing delay.**

Design a complete OpenCV-based system using face detection, feature extraction, recognition,

and tracking. Explain how you maintain identity across frames and prevent false or duplicate attendance records.

5. Smart Home Surveillance and Intrusion Detection

A home security system uses a fixed camera to monitor an entrance continuously.

The system should:

☐ **Detect significant motion.**

☐ **Distinguish actual movement from camera noise and small background changes.**

☐ **Track moving objects.**

☐ **Identify suspicious activity.**

☐ **Generate an alert when an unauthorized person enters a predefined region.**

☐ **Operate in real time with limited computational resources.**

Design an OpenCV-based surveillance pipeline using background subtraction, filtering,

morphological operations, contour/feature detection, motion analysis, and tracking.

Explain how

your approach reduces false alarms caused by shadows, illumination changes, trees, and other

dynamic background objects.

Pipeline:

Camera → Frame Capture → Background Subtraction → Filtering → Morphological Operations → Contour Detection → Motion Analysis → Object Tracking → Region Detection → Alert

Working

1. Frame Acquisition

- Capture continuous video using OpenCV VideoCapture().
- Resize frames to reduce processing time.

2. Background Subtraction

- Use **MOG2 or KNN** to separate moving objects from the background.
- Generates a foreground mask.

3. Filtering

- Apply **Gaussian/Median filtering** to reduce camera noise.
- Helps remove small unwanted changes.

4. Morphological Operations

- **Opening** removes small noise and isolated pixels.
- **Closing** fills gaps in detected objects.
- Produces a cleaner foreground mask.

5. Contour Detection

- Detect contours from the foreground mask.
- Ignore contours below a minimum area.
- Large contours are treated as possible moving objects.

6. Motion Analysis

- Compare object position and size across consecutive frames.
- Detect significant movement while ignoring very small changes.

7. Object Tracking

- Assign a unique ID to each moving object.
- Use **Centroid, CSRT, or KCF tracking** to follow objects across frames.

8. Intrusion Detection

- Define a **Region of Interest (ROI)** near the entrance.
- If a tracked person/object enters the restricted region, generate an alert.

Reducing False Alarms

- **Shadows:** Use morphological filtering and suitable foreground-mask thresholds.
- **Illumination changes:** Adaptive background models such as MOG2 update the background gradually.
- **Trees/leaves:** Ignore very small contours and objects with unstable movement.
- **Camera noise:** Gaussian/median filtering removes random noise.
- **Small movements:** Use a minimum contour-area and motion threshold.

Conclusion

The OpenCV pipeline detects significant motion, removes noise, tracks moving objects, and checks whether they enter a restricted region. **Background subtraction + filtering + morphology + contour analysis + tracking** reduces false alarms while maintaining real-time performance on limited hardware.